

SimGEA v.1.0.0 manual

Takahiro Sakamoto

June 30, 2026

1 Overview

SimGEA is a method for simulating neutral alleles that reflect the population structure observed in an input dataset. These simulated alleles can be analyzed using the same user-specified method as the empirical data, thereby generating a null distribution for calibrating p -values.

The workflow consists of two main steps: (i) estimation of a migration matrix and (ii) simulation of neutral alleles. The directories "mig_est" and "null_simu" contain the programs used for these respective steps.

The folder "calculate_p-GEA" contains Python scripts illustrating how the simulated neutral alleles can be used to calculate calibrated p -values. In this example, SimGEA is combined with a simple rank-biserial correlation test to assess the association between allele counts and an environmental variable. In principle, SimGEA can be used with any suitable association statistic or analysis method. The rank-biserial correlation workflow presented here provides a simple way to start a genotype-environment association analysis within the SimGEA framework.

2 Folder "mig_est" (estimation of a migration matrix)

2.1 Preparing an executable file

The code is written in C++. The Eigen library should be installed in advance. After moving to the folder "mig_est", the executable file can be compiled using the following command:

```
g++ -O3 -std=c++17 main.cpp my_matrix_calc.cpp options.cpp \  
-I /path/to/eigen \  
-o simgea_migration
```

Here, "/path/to/eigen" should be replaced with the path to the Eigen library.

2.2 Input file

The input file is expected to be a BayPass-style allele count dataset. The file should be a space-separated text file in which each line represents a SNP, and each population is represented by two integers corresponding to the counts of reference and alternative alleles. Missing data are not allowed; therefore, the total number of alleles in each subpopulation must be constant across loci. Sample sizes can differ among subpopulations. Users may apply an imputation method to deal with missing data before preparing the input file.

Example (10 subpopulations, each with 32 alleles):

```

26 6 32 0 32 0 25 7 32 0 32 0 28 4 30 2 31 1 30 2
30 2 31 1 27 5 31 1 27 5 31 1 27 5 32 0 31 1 31 1
14 18 7 25 4 28 3 29 2 30 8 24 12 20 8 24 6 26 2 30
22 10 31 1 29 3 21 11 25 7 31 1 26 6 32 0 29 3 31 1
4 28 5 27 9 23 9 23 1 31 5 27 8 24 6 26 8 24 10 22

```

2.3 Running the program

The compiled program can be run using the following command:

```
./simgea_migration -i <baypass_style_file>
```

The list of options is shown below:

```

-o, --out-prefix <STR>      Output prefix [out]
--maf <DOUBLE>              MAF filter in [0,0.5) [0.0]
--lr <DOUBLE>               AdamW learning rate [1e-3]
--beta1 <DOUBLE>            AdamW beta1 [0.9]
--beta2 <DOUBLE>            AdamW beta2 [0.999]
--weight-decay <DOUBLE>     AdamW weight decay [0.01]
--max-step <INT>             Max optimization steps [100000]
--min-error <DOUBLE>         Relative improvement threshold [1e-6]
--min-abs-error <DOUBLE>     Absolute improvement threshold [0.01]
--seed <UNSIGNED INT>        Random seed [randomly determined]
-l, --log-file <STR>         log file [log.txt]
-h, --help                   Show this help and exit

```

The default values are shown in square brackets.

An important point regarding the minor allele frequency filter is that it is based on the unweighted mean allele frequency across subpopulations, rather than on the pooled allele frequency across all individuals. To clarify this point, let us consider a locus at which a_i out of the b_i sampled alleles are derived alleles in the i th subpopulation ($i = 1, \dots, n$). In this case, the filter is applied based on $\frac{1}{n} \sum_i \frac{a_i}{b_i}$, rather than on $\frac{\sum_i a_i}{\sum_i b_i}$.

The program searches for a set of pairwise migration rates (or an $n \times n$ symmetric migration matrix, $\mathbf{M}$) that reproduces the covariance matrix estimated from the input data, $\mathbf{\Lambda}_{\text{obs}}$. Let $\mathbf{\Lambda}_{\text{theor}}(\mathbf{M})$ be a covariance matrix calculated from $\mathbf{M}$ using coalescent theory. The distance between these two matrices is evaluated by $D(\mathbf{\Lambda}_{\text{obs}}, \mathbf{\Lambda}_{\text{theor}}(\mathbf{M}))$, where $D(\mathbf{X}, \mathbf{Y}) = \sum_{i < j} (x_{ij} - y_{ij})^2$. The optimization process aims to reduce this distance.

Let $D(t)$ denote the value of the distance at optimization step t . The optimization is terminated when any of the following conditions is satisfied:

- After at least 1000 steps, ten successive steps satisfy $\frac{2D(t-1)-2D(t)}{2D(t-1)+\tau_1} < \tau_1$ and $2D(t-1) - 2D(t) < \frac{2D(0)-2D(1000)}{1000}\tau_2$.
- At $t = 1000k$ ($k \geq 2, k \in \mathbb{N}$), $\frac{2D(t-1000)-2D(t)}{2D(t-1000)+\tau_1} < \tau_1$.
- The total number of optimization steps reaches τ_3 .

Here, τ_1 corresponds to <min-error>, τ_2 corresponds to <min-abs-error>, and τ_3 corresponds to <max-step>.

2.4 Output files

"prefix_data_sample_mat.txt":	cov.matrix of sampled allele frequencies
"prefix_data_pop_mat.txt":	estimated cov.matrix of population allele frequencies
"prefix_est_mig_mat.txt":	estimated mig.matrix (2Nm, assuming N haploids)
"prefix_est_cov_mat.txt":	pop.cov.mat. calculated from the estimated mig.matrix
"prefix_est_sample_mat.txt":	sample cov.mat. calculated from the estimated mig.matrix

3 Folder "null_simu" (simulation of neutral alleles)

3.1 Preparing an executable file

The code is written in C++. After moving to the folder "null_simu", the executable file can be compiled using the following command:

```
g++ -O3 -std=c++17 simu.cpp options.cpp -o simgea_neutral
```

3.2 Input files / parameters

Three input files and one parameter are required.

<migration_file>:	est_mig_mat file produced in the "mig_est" folder
<baypass_style_file>:	baypass style data (used to obtain # of alleles in each subpop.)
<bin_file>:	a file defining bin breaks
<site_number>:	the number of simulated SNPs per bin

Optional parameters are as follows:

-o, --output <STR>	Output file [out_null.txt]
--n-pre <INT>	Number of trees in the prerun [10000]
--mut-density <DOUBLE>	Expected number of mutations per tree [0.25]
--seed <UNSIGNED INT>	Random seed [randomly determined]
-l, --log-file <STR>	Log file [log.txt]
-h, --help	Show this help and exit

This program runs structured coalescent simulations using the population-scaled migration rates specified in the migration file. The number of samples in each population is determined to match the specified BayPass style file. To enhance MAF-bin-based downstream analysis, this simulator generates the specified number of SNPs for each of the specified bins. The bin file should contain one number per line, representing the start of each bin. For example, a bin file

```
0.04999
0.075
0.1
0.125
0.15
0.175
0.2
0.225
0.25
0.275
```

0.3
0.325
0.35
0.375
0.4
0.425
0.45
0.475

represents 18 bins, consisting of $(0.04999, 0.075]$, $(0.075, 0.1]$, $\dots$, $(0.475, 0.5]$.

3.3 Running the program

The compiled program can be run using the following command:

```
./simgea_neutral -m <migration_file> -d <baypass_style_file> -b <bin_file> -s <site_number>
```

This program first generates a number of trees specified by the parameter n_{pre} . From these runs, the mutation rate is determined for each bin such that the expected number of mutations belonging to each bin and derived from a single tree is approximately equal to the parameter <mut_density> . After that, mutations are sampled while independent trees are generated until all bins contain the specified number of mutations. By doing so, mutations in each bin are sampled across approximately $\text{<site_number> / <mut_density>}$ trees. This aims to prevent many mutations from being sampled from the same tree, making linkage among the mutations unlikely.

3.4 Output files

<output file> : simulated allele count data

<bin file> : modified bin file

The allele count data follow the BayPass-style format, except that the first column represents the bin index. If the original bin file is not sorted or contains invalid bin values (i.e., values < 0 or ≥ 0.5), the values are sorted and the invalid values are removed. Bin indices are assigned based on the modified bin file, where the first bin corresponds to index 0, the second to index 1, and so on.

bin index of the first variant	allele counts in the 1st subpop.	allele counts in the 2nd subpop.	...
14	1 15	13 3	8 8 4 12 2 14 1 15 11 5 7 9 6
15	5 11 3 13	8 8 9 7 12 4 1 15 3 13 9 7 13	
16	1 15 14 2 9 7 7 9 3 13 9 7 14 2 8 8 5 1		
12	0 16 5 11 10 6 11 5 2 14 5 11 7 9 6 10		
0	3 13 0 16 3 13 3 13 0 16 2 14 0 16 0 16		

Figure 1: The format of the output file. The first column represents the bin index (starting from 0) and the subsequent columns follow the BayPass-style format.

4 Folder "calculate_p_GEA" (application to rank-biserial correlation)

4.1 Installing packages

The program in this folder is written in Python and requires the following packages:

- NumPy
- SciPy
- Numba

These can be installed using the following command:

```
python3 -m pip install numpy scipy numba
```

4.2 Input files

Four input files are needed.

```
<baypass_style_file>: empirical allele count data in a BayPass style
<env_file>:           BayPass style environment data
<null_file>:          simulated allele count data used for generating null distribution
<bin_file>:           a file defining bin breaks in the null file
```

In the above, the environment data are expected to be space-separated, with each row representing one environmental variable. Example (10 subpopulations, two environmental variables):

```
12.4 15.8 18.1 20.3 22.7 24.5 19.6 16.2 13.9 11.7
8.6 10.2 13.5 16.8 19.1 21.4 17.9 14.7 11.3 9.1
```

The list of options is shown below:

-o, --out-file <STR>	Output file [p_val.txt]
-l, --log-file <STR>	Log file [log.txt]
-q, --tail-quantile <DOUBLE>	Definition of the tail [0.995]
-h, --help	Show this help and exit

The default values are shown in square brackets.

4.3 Running the program

The program can be run using the following command:

```
python3 calculate_p_gea.py -b <bin_file> -e <env_file> -n <null_file> -d <baypass_style_file>
```

This program first evaluates genotype-environment associations using the simulated alleles. In this program, the rank-biserial correlation test (the rank of the environmental value vs. allelic presence or absence) is employed. Following Wendt (1972, *Eur. J. Soc. Psychol.*, 2(4): 463–465), this correlation is evaluated using the mathematical analogue of the Mann–Whitney U test.

From these results, the program constructs a null distribution of Z values for each bin. The distribution is smoothed and extrapolated using the following two fitting procedures. For the bulk of the distribution ($q < \text{<tail.quantile>}$), kernel density estimation (KDE) with Gaussian kernels is applied. For the tail ($q \geq \text{<tail.quantile>}$), a generalized Pareto distribution is fitted. The shape parameter of the generalized Pareto distribution is restricted such that the maximum possible Z value is within the support of the fitted distribution. This smoothed distribution is used to calculate p -values from the Z values obtained from genotype–environment associations observed in the empirical data.

4.4 Output file

<output file>: list of p -values

Each row represents one variant, and the p -values for different environmental variables are separated by spaces.

5 Example workflow

In this example, we assume that three input files have already been prepared directly under the folder "Sim_GEA".

```
"baypass_geno.txt": BayPass-style genotype file
"baypass_env.txt":  BayPass-style environmental file
"bin.txt":          a file defining bin breaks in the bin analysis
```

In the following code, it is assumed that the Eigen library is already in the C++ compiler's include path.

The following commands run the entire analysis.

```
cd mig_est
g++ -O3 -std=c++17 main.cpp my_matrix_calc.cpp options.cpp -o simgea_migration
./simgea_migration -i ../baypass_geno.txt

cd ../null_simu
g++ -O3 -std=c++17 simu.cpp options.cpp -o simgea_neutral
./simgea_neutral -m ../mig_est/out_est_mig_mat.txt -d ../baypass_geno.txt -b ../bin.txt -s 100000

cd ../calculate_p_GEA
python3 calculate_p_gea.py -b ../bin.txt -e ../baypass_env.txt -n ../null_simu/out_null.txt \
    -d ../baypass_geno.txt
mv p_val.txt ../
```